aFluid: A modular framework for robots learning in fluids.

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—The abstract goes here.**

I. Introduction

Learning-based controllers have expanded what robots can do. Much of this progress comes from online reinforcement learning (RL) in massively parallel simulation, where thousands of robots generate training experience at once [4, 6]. The supporting software is largely built around joint actuators for humanoids, quadrupeds, and manipulators. Robots that fly, sail, or swim need the same support for actuators and fluid effects that apply forces and torques directly to their bodies.

Learning through trial and error on hardware is costly for these robots. Flight experiments risk crashes, and underwater trials require deployment and recovery in conditions that are difficult to repeat [1]. Simulation provides a controlled setting for training and testing before deployment. A shared framework would also make it easier to carry physical models and training tools from one vehicle to another.

Fast simulation is useful only if it captures the physics that matters for control. Propellers take time to respond to commands, wing lift changes with airspeed, and buoyancy changes as a vehicle enters water. A policy trained without these effects may rely on behavior that the real robot cannot reproduce. More detailed models cost more to evaluate, however, and a model that is fast enough for one vehicle can become too expensive when repeated across thousands of training environments.

Stonefish provides detailed marine dynamics and sensors and can run separate simulations in parallel [1]. For aerial robots, Flightmare separates quadrotor dynamics from rendering [7], while Aerial Gym runs multirotor dynamics and controllers in parallel on the GPU [3]. These simulators already support learning within their target domains. We aim to make physical models reusable across aircraft, surface vessels, and underwater vehicles within the same learning framework.

Isaac Lab and Newton provide standard interfaces for joint actuators [6, 8]. Their physics engines also accept body forces and torques, but the models that produce them must be connected to commands, body motion, and fluid conditions. Actuator state has to be updated at the correct simulation step and cleared or restored when an episode resets. With many environments, a reset must affect only the selected robot. If each task handles these details itself, moving a model to a new vehicle means rewriting the surrounding code.

We present **aFluid**, a modular framework for learning to control robots in fluids. Built on Newton, aFluid provides a library of analytical actuator and fluid models, a shared GPU runtime, and an Isaac Lab integration with reference learning tasks. The runtime handles model updates and resets across environments. Users can choose or extend the models a vehicle needs while keeping the same tools for training, randomization, and evaluation.

Models of propellers, lifting surfaces, and buoyancy all produce forces and torques on a body. aFluid uses this shared output as the interface between a physical model and the simulator. Each component returns a wrench—a force and torque—in the same coordinate frame at the body's center of mass, so their contributions can be added. Fluid properties are supplied separately from the force calculation; the same lift model can therefore describe a sail in air or a rudder in water. The components run in batches on the GPU, with common rules for updates and resets. Newton advances the robot using their combined forces and torques, while the task code uses the same interfaces regardless of the chosen models.

In preliminary simulation experiments, we train policies for quadrotor control, flips with recovery, assisted fixed-wing navigation, and sailboat control in up to 4,096 parallel environments. In evaluations with new random seeds, the policies complete 254 of 256 flip-and-recovery maneuvers, all 256 fixed-wing waypoint courses, and 232 of 256 sailboat courses under randomized steady wind. These tasks use different actuators and fluid models but share the same simulation and training interfaces.

II. Section

Section text here.

A. Subsection Heading Here

Subsection text here.

1) Subsubsection Heading Here: Subsubsection text here.

III. RSS citations

Please make sure to include `natbib.sty` and to use the `plainnat.bst` bibliography style. `natbib` provides additional citation commands, most usefully `\citet`. For example, rather than the awkward construction

```
\cite{kalman1960new} demonstrated...
```

rendered as "[2] demonstrated...," or the inconvenient

```
Kalman \cite{kalman1960new}
demonstrated...
```

rendered as "Kalman [2] demonstrated...", one can write

```
\citet{kalman1960new} demonstrated...
```

which renders as "Kalman [2] demonstrated..." and is both easy to write and much easier to read.

A. RSS Hyperlinks

This year, we would like to use the ability of PDF viewers to interpret hyperlinks, specifically to allow each reference in the bibliography to be a link to an online version of the reference. As an example, if you were to cite "Passive Dynamic Walking" [5], the entry in the bibtex would read:

```
@article{McGeer01041990,
  author = {McGeer, Tad},
  title = {\href{http://ijr.sagepub.com/conten
  t/9/2/62.abstract}{Passive Dynamic
  Walking}},
  volume = {9},
  number = {2},
  pages = {62-82},
  year = {1990},
  doi = {10.1177/027836499000900206},
  URL = {http://ijr.sagepub.com/content/9/2/62
  .abstract},
  eprint = {http://ijr.sagepub.com/content/9/2
  /62.full.pdf+html},
  journal = {The International Journal of
  Robotics Research}
}
```

and the entry in the compiled PDF would look like:

[1] Tad McGeer. Passive Dynamic Walking. *The International Journal of Robotics Research*, 9(2):62–82, 1990.

where the title of the article is a link that takes you to the article on IJRR's website.

Linking cited articles will not always be possible, especially for older articles. There are also often several versions of papers online: authors are free to decide what to use as the link destination yet we strongly encourage to link to archival or publisher sites (such as IEEE Xplore or Sage Journals). We encourage all authors to use this feature to the extent possible.

IV. Conclusion

The conclusion goes here.

Acknowledgments

References

[1] Michele Grimaldi, Patryk Cieślak, Eduardo Ochoa, Vibhav Bharti, Hayat Rajani, Ignacio Carlucho, Maria Koskinopoulou, Yvan R. Petillot, and Nuno Gracias. Stonefish: Supporting machine learning research in marine robotics, 2025. URL https://arxiv.org/abs/2502.11887.

[2] R.E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82 (1):35–45, 1960.

[3] Mihir Kulkarni, Welf Rehberg, and Kostas Alexis. Aerial Gym Simulator: A framework for highly parallelized simulation of aerial robots. *IEEE Robotics and Automation Letters*, 2025. URL https://arxiv.org/abs/2503.01471.

[4] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac Gym: High performance GPU-based physics simulation for robot learning, 2021. URL https://arxiv.org/abs/2108.10470.

[5] Tad McGeer. Passive Dynamic Walking. *The International Journal of Robotics Research*, 9(2):62–82, 1990. doi: 10.1177/027836499000900206. URL http://ijr.sagepub.com/content/9/2/62.abstract.

[6] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, et al. Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025. URL https://arxiv.org/abs/2511.04831.

[7] Yunlong Song, Selim Naji, Elia Kaufmann, Antonio Loquercio, and Davide Scaramuzza. Flightmare: A flexible quadrotor simulator. In Jens Kober, Fabio Ramos, and Claire Tomlin, editors, *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 1147–1157. PMLR, 2021. URL https://proceedings.mlr.press/v155/song21a.html.

[8] The Newton Contributors. Newton: GPU-accelerated physics simulation for robotics and simulation research. Software, 2025. URL https://github.com/newton-physics/newton.